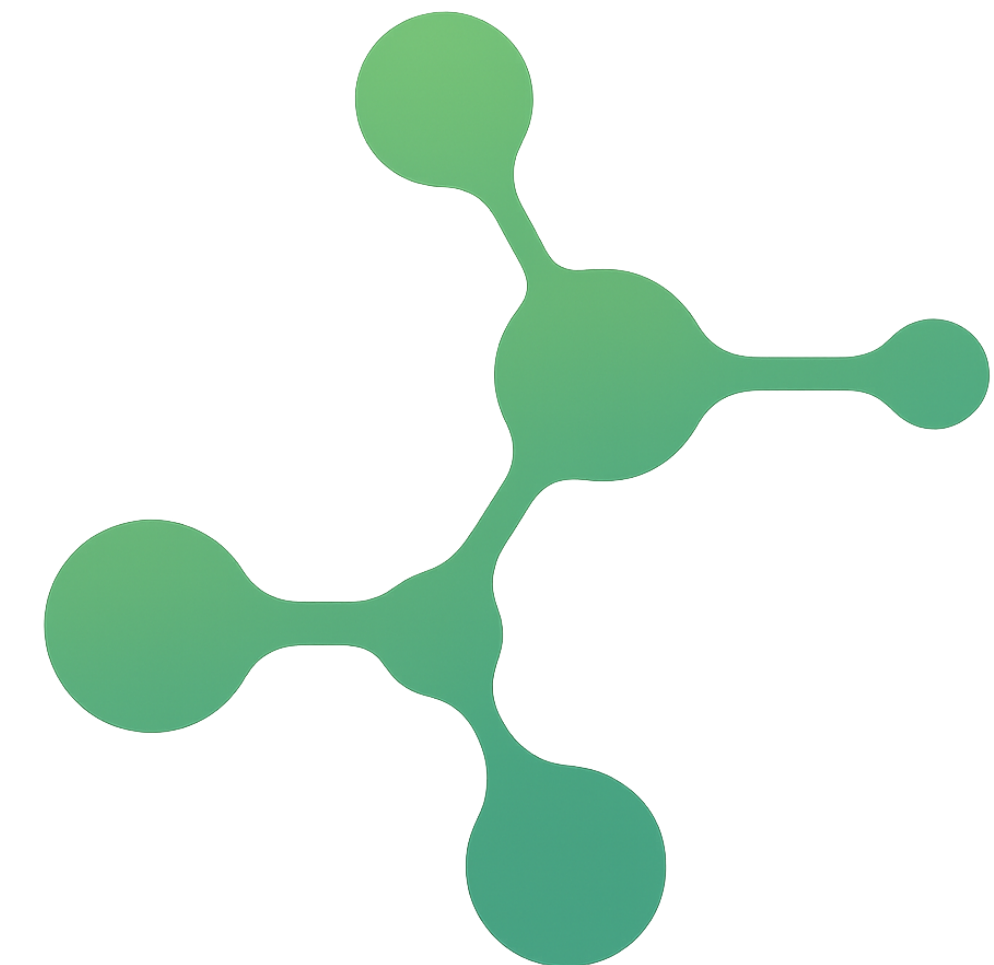


AbyssEye 初回利用ガイド

初回利用者向け / ROI抽出・DeepScan・リアルタイムエンジンの基本操作
この資料では、AbyssEye を初めて使う人が迷いやすいポイントを

Home から結果出力まで順番にまとめています。



1 起動

2 入口選択

3 確認と修正

4 PDF / CSV出力

推奨 branch: AbyssEye-Neo

- ・ branch は AbyssEye-Neo を使う

1. 起動前の準備

READMEの手順に沿って backend と frontend を起動します。

- ・ backend と frontend は別々に起動

- ・ 起動後は `http://localhost:3000` を開く

- ・ API は通常 `http://localhost:8000/api/v1/` を使う



Terminal 1

```
git clone -b AbyssEye-Neo https://github.com/Gashu884/AbyssEye.git
cd AbyssEye
python -m venv venv
source venv/bin/activate
pip install -r backend/requirements.txt
python backend/main.py
```

Terminal 2

```
cd frontend
npm install
npm run dev
```

Browser

`http://localhost:3000`

※ Python 3.14 では TensorFlow の install が通らないことがあります。

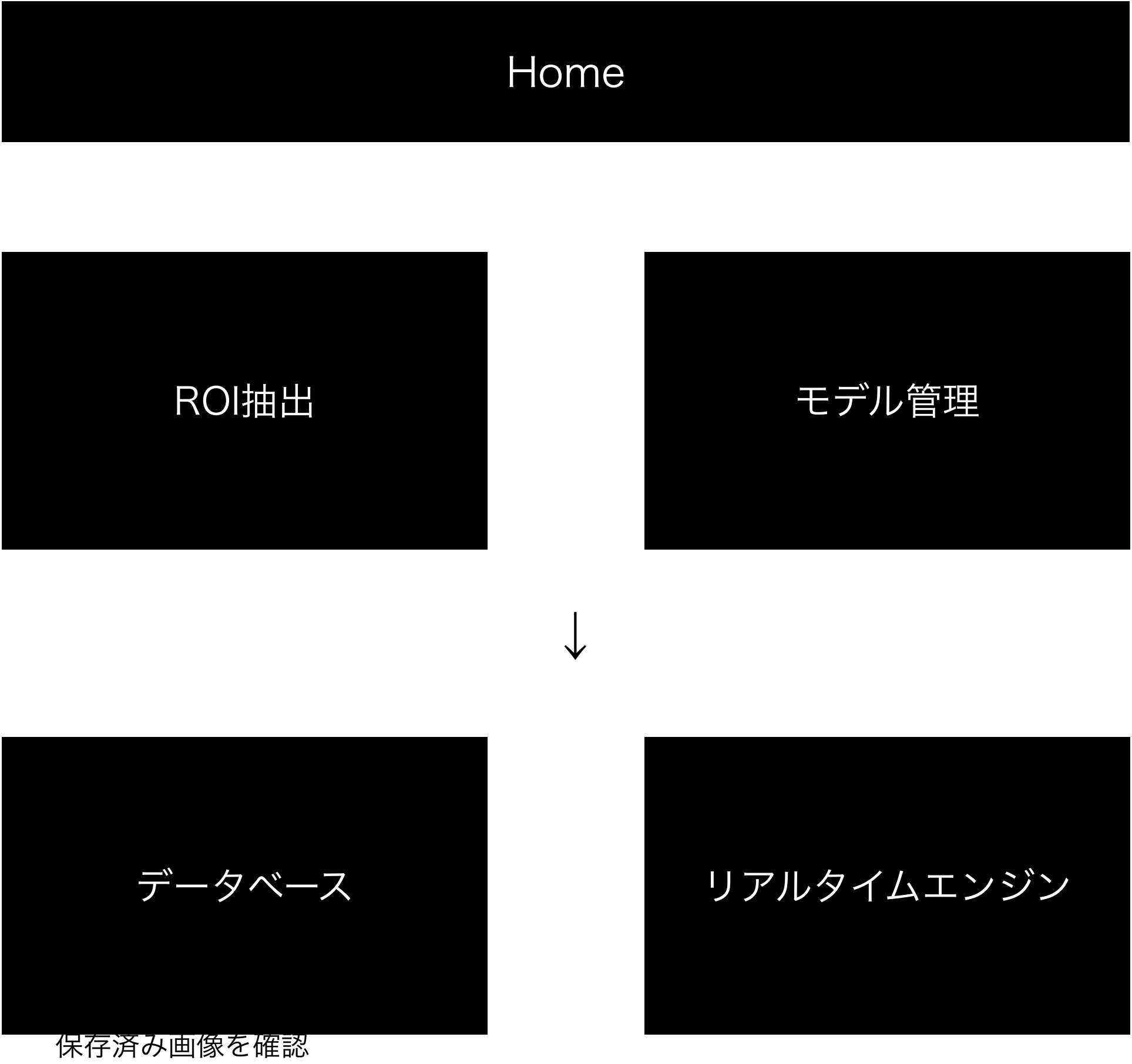
- ・ Home では ROI抽出 と モデル管理 が並ぶ

2. Home と 入口の選び方

Home の ROI抽出 押す 目的はなに？ 入口を選びます。

- ・ ROI抽出を押すとデータベースとリアルタイムエンジンに分かれる

- ・ 保存済み画像をまとめて扱うなら データベース
- ・ 最新 TIFF を見ながら保存するなら リアルタイムエンジン



- ・上段が画像リスト、下段が同視野フォルダリスト
- ### 3 データベース画面の使い方
- プロジェクト単位で画像と同視野画像ファイルを整理します。

- ・単一画像は ROI抽出&推論・DeepScan・保存・セルカウントへ進む

- ・同視野フォルダは 一覧 / マージ画像ROI抽出 を使う



プロジェクト作成 / 検索 / 画像アップロード / 同視野画像ファイルをアップロード

画像リスト
- ROI抽出&推論
- DeepScan
- 保存
- セルカウント

同視野フォルダリスト
- 一覧
- マージ画像ROI抽出

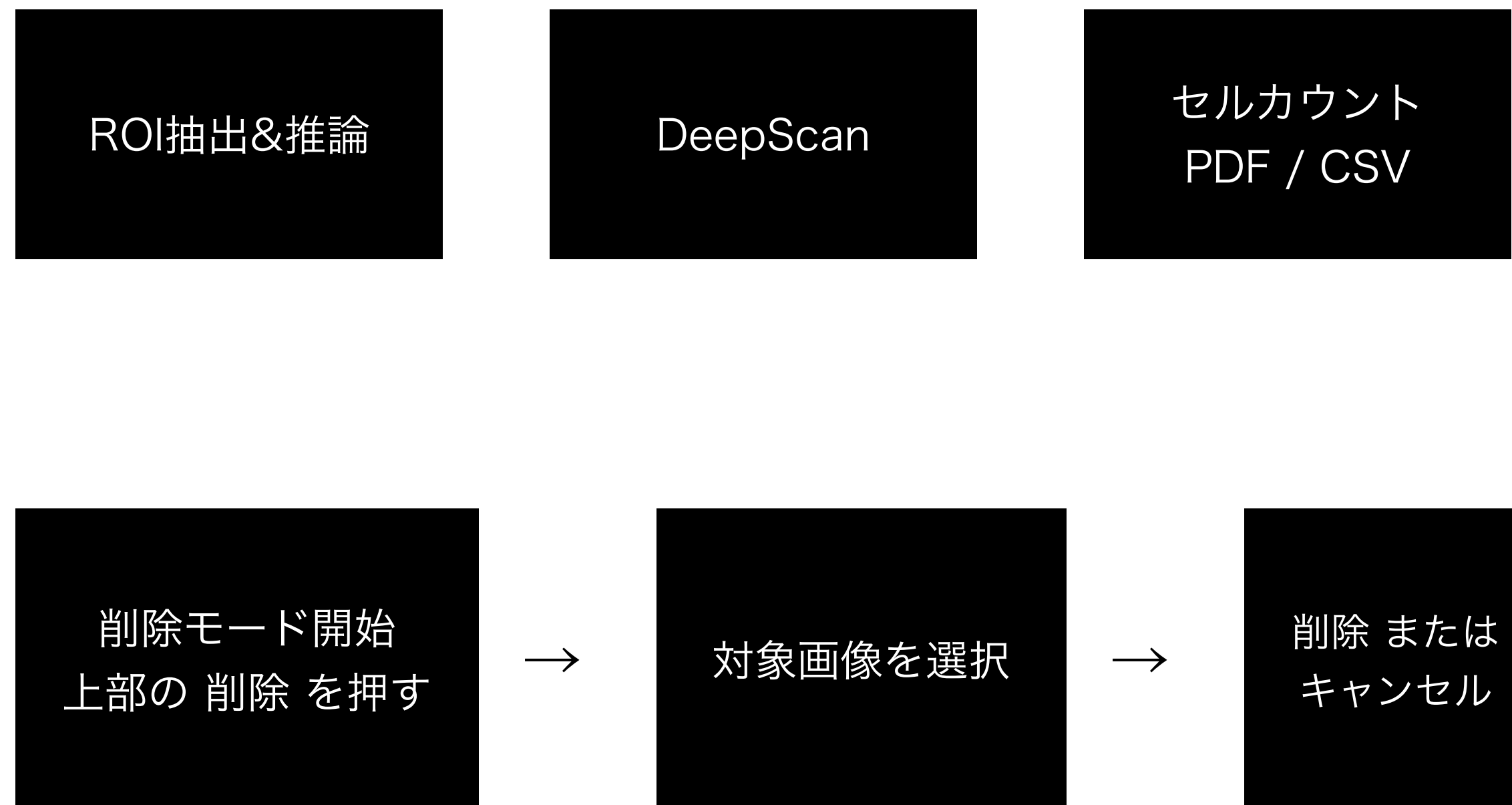
パンくずは Home > ROI抽出 > データベース > [各プロジェクト] で統一

- ・削除は 2 段階方式

4. 画像リストで行う操作

単一画像の処理と安全な削除は、この一覧から進めます。

1. 上部の 削除 を押す
2. 削除モードで対象画像を選ぶ
3. 削除 または キャンセル



各行の直接削除ではなく、上部操作に統一して誤操作を防ぎます。

- ・ 右側で AI / Manual の表示基準を切り替える

5. DeepScan の見方を更新できる

ROI Overlay を確認しながら、手動ROIと手動ROIを調整します。

- ・ 手動ROI追加 / 手動ROI削除 が使える

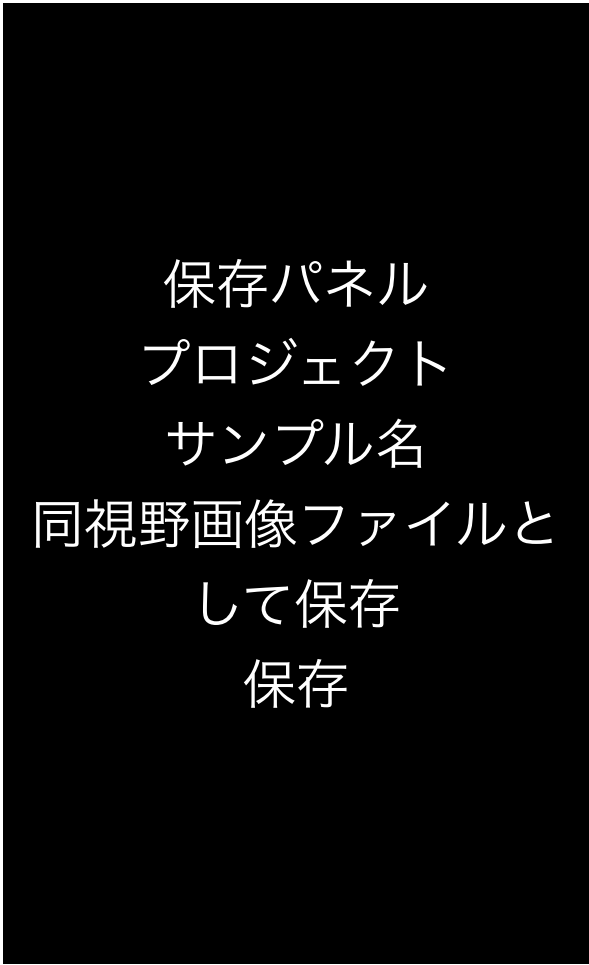
- ・ ROI移動 を ON にすると、約 0.45 秒長押し後ドラッグで位置更新



・同視野画像ファイルとして保存 を ON にすると、同一視野の最新 TIFF を見ることができ、そのまま DeepScan で確認できます。

・保存した画像は後から DeepScan で再確認できる

・AI / Manual 表示、manual label、手動 ROI の考え方は DeepScan と共通



説明文の表示が変わっても、ROI overlay 自体はずれないように調整済みです。

- ・ ROI 表は 画像順 / 信頼度順 / ROI番号順 で並べ替え可能

7. 結果の出力と更新

DeepScan 結果は PDF / CSV に保存し、利用者側は ff-only で更新します。

- ・ CSV は image_name 中心の列構成で、manual_label は 1 列に整理

- ・ 更新時は git pull --ff-only origin AbyssEye-Neo を使う

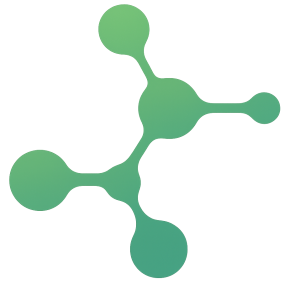
JAMSTEC_roi_labels (2)																
project_name	source_id	db_name	image_name	roi_id	ai_label	manual_label	final_label	label_source	model_confidence	manual_added	bbox_xmin	bbox_ymin	bbox_xmax	bbox_ymax	image_width_px	image_height_px
JAMSTEC	CK0606_1H1_20251205_001	JAMSTEC_CK0606_1H1_20251205_001_bulk.db	CK0606_1H1_20251205_001.tif	1	3			3 ai	0.8888468742370610	FALSE	229	511	277	559	1440	900
JAMSTEC	CK0606_1H1_20251205_001	JAMSTEC_CK0606_1H1_20251205_001_bulk.db	CK0606_1H1_20251205_001.tif	2	0			0 ai	0.9530859589576720	FALSE	911	765	959	813	1440	900
JAMSTEC	CK0606_1H1_20251205_001	JAMSTEC_CK0606_1H1_20251205_001_bulk.db	CK0606_1H1_20251205_001.tif	3	0			0 ai	0.9962230920791630	FALSE	456	852	504	900	1440	900
JAMSTEC	CK0606_1H1_20251205_001	JAMSTEC_CK0606_1H1_20251205_001_bulk.db	CK0606_1H1_20251205_001.tif	4	0			0 ai	0.9988433122634890	FALSE	844	462	892	510	1440	900
JAMSTEC	CK0606_1H1_20251205_001	JAMSTEC_CK0606_1H1_20251205_001_bulk.db	CK0606_1H1_20251205_001.tif	5	0			0 ai	0.9991455078125	FALSE	563	80	611	128	1440	900

CSV 出カイメージ

- つまずきやすい点
- ・ DeepScan が開かない → 先に ROI抽出&推論
 - ・ install エラー → Python バージョン確認

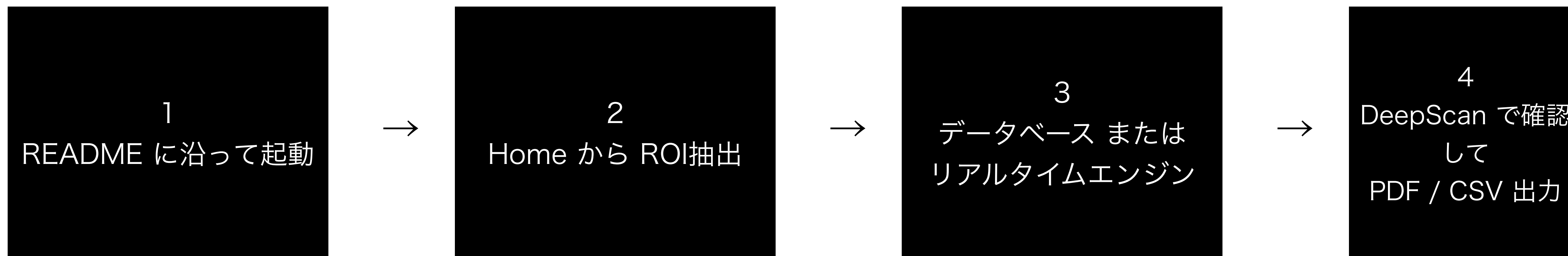
出力
PDF出力
CSV出力
クラス別確認

更新
git checkout AbyssEye-Neo
git pull --ff-only origin AbyssEye-Neo



8. まず触るならこの順番

初回は次の4ステップだけ覚えておくと便利です。



最初は データベース → ROI抽出&推論 → DeepScan → セルカウント の流れがいちばん分かりやすいです。

慣れてきたら、リアルタイムエンジンで保存しながら確認する運用に広げてください。

